二、Model+Input+Output

1.Model I/O： Model

在成功申请大模型 API 并能够发送基本请求之后，我们还需要了解一个关键问题：**如何控制模型的生成行为**？同样的输入，模型有时给出确定答案，有时表现出创造性；有时输出简洁，有时冗长——这些差异主要源于一组可配置的**生成参数**。

1.1 必填参数

参数名称	类型	说明	示例值
API密钥	String	身份验证凭证，必填参数	`your_api_key`
模型名称	String	指定模型版本（如 `deepseek-chat`）	`model="deepseek-chat"`
输入文本	String/List	用户输入或对话上下文（支持多轮对话）	`messages=[{"role":"user","content":"你好"}]`

1.2 核心参数

1.2.1 核心参数1：最大生成长度

`max_tokens`（或 `max_length`）是最基础且通常为必填的生成参数。它用于**限制模型在一次调用中最多生成的 token 数量**。

1.能力界定：设定的是“天花板”，而非“保证值”

该参数的作用是给出一个**硬性的上限**：无论模型多么“想”继续写下去，一旦生成的 token 数达到该阈值，生成过程会立即截断并返回当前内容。

但需要注意：**实际返回的内容长度常常小于该上限。**原因包括：

- 模型输出了特殊的**结束符**（如 `<|endoftext|>`、`</s>`），主动停止了生成。
- 模型在生成过程中自然完成了句子或段落，虽然没有输出结束符，但某些 API 会设置隐式的停止条件（如检测到句号后不再强制继续）。
- 模型内部概率分布极度偏向某个较短的路径，提前达到了“逻辑终点”。

因此，`max_tokens` 更像一个 **“安全阀”**：它防止模型无休止地生成导致资源耗尽，但不保证每次都能用完这个额度。

2.为什么它是必填参数?

大多数 API 要求必须显式提供 `max_tokens`，原因有二：

1. **成本控制**：按 token 计费的模式下，没有上限可能导致恶意或意外的超长生成，产生无法预估的费用。
2. **资源管理**：服务端需要为每次请求预先分配显存和计算资源，知道最大可能的输出长度有助于优化调度。

3.设置建议

将 `max_tokens` 设为比期望输出长度略大的一个保守上限（例如期望50 token，设100），真正的长度控制交给提示词中的具体要求（如'一句话'、'200字以内'、'只输出数字'）。`max_tokens` 只用于防止模型失控，不用于精调长度。

4.Token估算：

在调用 API 前，可用以下换算快速估算文本的 token 数：

- **英文**：1 token ≈ 4 个英文字符（含空格）。

  例如：`"Hello world"` 11 个字符 → 约 3 token。

  更粗略：1 个英文单词 ≈ 1.3 token。
- **中文**：1 token ≈ 1.5 ~ 1.6 个中文字符。

  例如：`“你好”` 2 个字符 → 约 1.3 token；`“人工智能”` 4 字符 → 约 2.5 token。

  为方便心算，通常取 **1 个汉字 ≈ 0.67 token**（即 3 个汉字 ≈ 2 token）。
- **中英文混合**：分别按上述规则累加。
- **标点、数字、空格**：英文字符按 4 字符/token，中文标点按中文字符估算。

5.Token实际计算

`tiktoken` 是 OpenAI 开源的 Python 库，**完全在本地运行**，不依赖 API 调用。

- **安装时**：从 PyPI 下载（国内正常访问，若慢可换清华源），**不需要翻墙。**
- **运行时**：首次使用时，它会从 GitHub 下载一个编码表文件（`cl100k_base.tiktoken`）

这个会下载在缓存《.cache》文件夹里面，可以直接使用上传的包。

（1）安装tiktoken

```
pip install tiktoken
```

（2）使用示例

```
import tiktoken

# 选择编码器（GPT-4 / GPT-3.5-turbo 使用 cl100k_base）
enc = tiktoken.get_encoding("cl100k_base")

text = "大模型在日常对话中的表现已经相当成熟"
tokens = enc.encode(text)

print(len(tokens))          # 输出 token 数量
print(tokens)               # 输出 token ID 列表（可选）
```

注：这里由于网络问题，需要科学上网，否则可以下载课件附带的压缩文件夹，具体操作如下：

（3）本地配置（可选）

```
import os
os.environ["TIKTOKEN_CACHE_DIR"] = r"D:\my_class\tiktoken_cache"  # 替换成你解压的路径
```

设置好环境变量后，后面代码就一样了。

1.2.2 核心参数2：温度

在前文中，我们介绍了 `max_tokens` —— 它规定了模型生成内容的最大长度，相当于给模型的“发言”划定了上限。然而，长度只是生成行为的一个维度。更关键的问题是：**模型在每一步应该以多大的“自由度”选择下一个词**?

这正是温度参数所控制的核心。温度参数的计算公式涉及对模型输出概率分布的调整，其核心原理是通过缩放模型的**logits**（未归一化的预测值）来改变生成文本的随机性和多样性。

1.温度参数的计算公式

温度参数 T 作用于模型的 logits z_i，调整后的概率分布公式为：

$$P'(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

公式含义：

- z_i：模型对第 i 个 token（词汇或标记）的原始预测值（logits）。
- T：温度参数，控制概率分布的“尖锐”或“平滑”程度。
- $P'(x_i)$：调整后的概率分布，决定模型生成每个 token 的可能性。

2. 温度对分布的影响

温度设置	分布形态	生成行为	适用场景
低温T→0	尖锐 peak	几乎总选最高概率 token，输出确定、可重复	事实问答、代码生成、分类
默认 T=1	原始分布	保持模型原有概率	通用对话
高温 T>1	平滑 flat	低概率 token 也有机会被选中，输出多样、创意	诗歌生成、头脑风暴

注意：T=0 在数学上需特殊处理（除零），实际 API 中通常将其视为**确定性解码**（直接取 argmax）。

3. 数值示例

假设模型对下一个 token 的 logits 为 z=[2.0,1.0,0.1]，对应词汇 `["蓝色", "灰色", "红色"]`。不同温度下的概率分布变化如下：

温度 T	概率分布 P' 近似	生成倾向
0.5	[0.86, 0.12, 0.02]	几乎必然选“蓝色”

温度 T	概率分布 P' 近似	生成倾向
1.0	[0.66, 0.24, 0.10]	大概率“蓝色”，偶有“灰色”
2.0	[0.50, 0.30, 0.20]	“蓝色”优势减弱，低概率词出现机会增加

该示例直观表明：**温度越低，分布越集中；温度越高，分布越均匀。**

4.常见误区

很多人认为“温度越高，模型回答的逻辑性越差，甚至胡言乱语”。

（1）**正解**

温度参数控制的本质是**随机性的大小**，而非逻辑性。高温下，模型只是更愿意探索那些原本概率较低的词——这些词在概率排序中处于靠后位置，但未必不符合语法或语义。模型在每一步依然从完整的概率分布中采样，只要温度不是极端高（例如超过2），被选中的词仍属于模型认为“合理”的词汇集合。

- 低温（趋近0）强制模型每次都选择概率最高的词，输出结果高度确定、可复现。
- 高温允许低概率词被选中，输出结果呈现多样性，但单次输出中的每个词仍是在当前上下文下模型认为合理的候选。

（2）**关键应用理解**

对于多步骤推理或计算类任务，必须使用低温。

原因在于：这类任务要求每一步都严格遵循正确的逻辑路径，任何偏离（即使选择了语义上合理但与正确推理方向不一致的词）都会导致后续步骤的连锁错误。

低温消除了这种“偏离”的可能性，保证了每一步的输出都是模型认为最合理的选项。

反之，高温引入的随机性虽然不直接等同于“逻辑错误”，但在需要精确链式推理的场景中，它会显著增加答案出错的概率。

5.对比

```python
from root import llm

user_input = "今天天气不错，_____"

prompt = f"""
任务（Task）：续写句子
内容（Context）：{user_input}
约束（Constraint）：生成内容必须在 50 个字符以内
"""

# Temperature = 0
print("=== Temperature = 0 ===")
for i in range(3):
    result = llm.invoke(prompt, temperature=0)
    print(f"\n[第 {i+1} 次]")
    print(result)

```

```python
# Temperature = 2
print("\n=== Temperature = 2 ===")
for i in range(3):
    result = llm.invoke(prompt, temperature=2)
    print(f"\n[第 {i+1} 次]")
    print(result)
```

2.2.3 核心参数3：核采样

`top_p`（又称核采样）用于动态决定模型每一步生成时考虑的候选词集合。它不固定候选词数量，而是根据概率分布的集中程度自动调整。

1. 核心原理

设定一个累积概率阈值 p（通常 0.7~0.95），模型按以下步骤操作：

1. 将所有候选词按概率从高到低排序。
2. 从最高概率词开始，依次累加其概率，直到累积和 **首次 ≥ p**。
3. 将这些被累加的词作为候选集，其余低概率词被丢弃。
4. 在候选集内重新归一化概率，然后进行随机采样。

示例：假设某步各词概率为 `[0.5, 0.3, 0.1, 0.05, 0.03, ...]`，设置 $p = 0.9$：

- 累加：0.5 → 0.8 (<0.9) 继续；加 0.1 → 0.9 (≥0.9) 停止。
- 候选集包含前三个词（概率分别为 0.5, 0.3, 0.1）。
- 这三个词的概率被重新归一化（总和为 0.9），变成 `[0.556, 0.333, 0.111]`，然后随机采样。

2. p 值的效果

p 值	候选集大小	生成行为
小（如 0.1~0.3）	很小（通常 1~2 个词）	输出确定、稳定，几乎每次选同一个词
中（如 0.5~0.7）	适中（几个到十几个词）	有一定多样性，但不会选极低概率词
大（如 0.9~1.0）	较大（可能包含几十甚至上百个词）	多样性强，可能选出罕见但合理的词

3. 如何理解不同 p 值的区别

我们现在分别使用不同的top_p值，让大模型给我们随机推荐颜色：

```python
from root import llm

prompt = """
任务（Task）：随机推荐一种颜色
约束（Constraint）：只生成颜色名称，不要添加任何解释或额外文字
示例（Few-shot）：红色
"""

for p in [0.1, 0.5, 0.8]:
```

```
    print(f"\n现在的 top_p 值是：{p}")
    print("=" * 30)

    for i in range(10):
        result = llm.invoke(prompt, top_p=p)
        print(f"LLM推荐的颜色是：{result}")
```

4.与 `temperature` 的搭配使用

- **温度** 先调整概率分布的“尖锐”或“平坦”程度。
- **top-p** 再在调整后的分布上划定候选词范围。

`deepseek`官方推荐（之前的推荐）：

任务类型	Top-p值	温度参数	效果
创意写作/诗歌生成	0.9~0.95	1.0~1.5	增加新颖表达和创造力，避免逻辑混乱。
代码生成/事实问答	0.7~0.8	0.2~0.7	提高准确性和稳定性，减少错误。
对话系统	0.85~0.9	0.7~1.0	保持流畅和自然，避免重复或无关内容。
内容摘要	0.8~0.9	0.5~0.8	生成简洁、准确的摘要，避免冗余。

1.3 其他参数

1.3.1 top_k：固定数量的候选词截断

`top_k` 参数用于**限定模型每一步生成时考虑的候选词数量**。与动态调整的 `top_p` 不同，`top_k` 采用固定数量的截断方式：无论当前概率分布如何，模型仅保留概率最高的前 k 个词作为候选集，其余词被直接忽略，然后在此候选集内重新归一化概率并进行随机采样。

效果与取值建议

- **k 值较小（如 5~10）**：候选集狭窄，输出确定性高，适合需要严格控制的场景（如实体抽取、格式约束）。
- **k 值中等（如 20~50）**：保留适度灵活性，同时过滤掉极低概率的噪音词，是多数任务的常用范围。
- **k 值较大（如 100 或全部）**：接近不使用此参数，保留几乎所有词汇。

注意：不同 API 对 `top_k` 的支持程度不同。部分服务（如 OpenAI）并未直接开放 `top_k`，而是推荐使用 `top_p`；而某些开源模型或本地部署框架则同时支持两者。

1.3.2 stop：停止序列

`stop` 参数用于**指定一个或多个字符串作为生成终止信号**。当模型生成的文本中**首次出现**任一停止序列时，生成过程立即结束，且该停止序列本身**不会**被包含在最终输出中。

典型用法

使用场景	停止序列示例	说明
单句回答	`["。", "\n"]`	遇到句号或换行即停止
结构化输出	`["}", "]"]`	配合 JSON 或数组格式
多轮对话	`["用户：", "\n\n"]`	防止模型继续扮演用户
角色扮演	`["\n角色B："]`	在对话角色切换点截断

与 `max_tokens` 的配合

- `stop` **优先于** `max_tokens`：若两者同时设置，`stop` 先被检查，匹配成功则即时停止；仅当全程未匹配任何停止序列时，`max_tokens` 作为硬性上限生效。
- **建议**：将 `max_tokens` 设为略大于预期输出的值，作为防无限生成的安全阀，将精确截断交给 `stop`。

代码示例：

```
from root import llm

prompt = """
任务（Task）：生成一篇科幻文章
内容（Context）：主人公名叫 Alice
约束（Constraint）：文章长度约 500 字
"""

result = llm.invoke(prompt, stop=['。'])
print(result)
```

1.3.3 输出方式

流式输出（Streaming）与非流式输出（Non-Streaming）

- **非流式输出（全部输出）**：模型完整生成全部回答后，一次性返回整个文本。用户需等待生成完全结束才能看到结果，总等待时间较长，但实现简单，适用于离线任务或对延迟不敏感的场景。
- **流式输出**：模型每生成一个 token（或一小段）即立刻发送给客户端，用户可实时看到逐个字出现的效果。首字响应极快，交互感强，适用于聊天机器人、实时翻译等对延迟敏感的任务。
- **实现差异**：流式输出需在 API 请求中设置 `stream=True`，服务端以 SSE（Server-Sent Events）格式分块传输；非流式则为普通 HTTP 请求，等待完整响应后解析 JSON。

1.3.4 延迟

TTFT（Time to First Token，首见延迟）

- **定义**：从发送请求到收到**第一个 token** 的时间间隔。
- **意义**：直接反映模型的**首字响应速度**，是评估用户体验的关键指标。TTFT 短（如 < 200ms）能让用户感觉模型“开始思考了”，显著降低感知延迟。

- **影响因素**：输入长度（长输入需要更多预填充时间）、服务端负载、推理引擎效率、网络往返延迟。

E2E Latency（End-to-End Latency，端到端延迟）

- **定义**：从发送请求到**完整接收全部生成内容**所经历的总时间。
- **与 TTFT 的关系**：E2E 延迟 = TTFT + 剩余 token 的生成与传输时间。对于长文本生成，E2E 延迟主要取决于生成速度和输出长度；对于短问答，TTFT 往往占主导。
- **适用评估**：E2E 延迟用于衡量整体处理效率，在批处理或离线任务中更为重要；而流式输出虽然不影响 E2E 总耗时，但能通过提前显示部分内容改善交互体验。

计算 E2E Latency

```python
import time

t1 = time.time()
time.sleep(3)
t2 = time.time()
delta_t = t2 - t1
print(f'一共经过了{delta_t:.1f}秒')
```

1.3.5 参数封装

我们把上面要修改的参数，统一到模型封装里面

```python
from openai import OpenAI
import time
class Model:
    def __init__(self):
        self.model_name = 'deepseek-chat'
        self.base_url = "https://api.deepseek.com"
        self.client =OpenAI(api_key=api_key, base_url=self.base_url)

    def invoke(self,prompt,max_tokens = 1000,temperature = 0.9,top_p = 0.95,
            stop = None):
        time1 = time.time()
        if not isinstance(prompt,list):
            # 如果prompt不是列表，则将其转换为列表
            prompt = [
                {"role":"system","content":"你是一个有帮助的助手"},
                {"role": "user", "content": prompt}
            ]
        response = self.client.chat.completions.create(
            model="deepseek-chat",  # chat：不带深度思考的，reasoner：带深度思考的
            messages=prompt,  # 对话消息列表
            max_tokens=max_tokens,
            temperature=temperature,
            top_p=top_p,
            stream=False  # 是否流式返回结果,一般默认非流式
        )
        time2 = time.time()
```

```
        print(f'推理耗时：{time2 - time1:.2f}秒')
        return response.choices[0].message.content
llm = Model()
```

2.Model I/O：I

在 LangChain 的 Model I/O 模块中，**I（Input，输入）**的核心组件就是**提示词模板。**它的作用是将提示词的固定结构（如系统指令、输出格式要求）与动态变量（如用户问题、上下文）分离。开发者先定义模板框架，运行时再注入具体变量生成完整提示词。

这样做的好处是：避免手动拼接字符串带来的错误和重复劳动，提高代码的可维护性与复用性，为后续链式调用奠定基础。

2.1 PromptTemplate

2.1.1 场景假设：天气查询助手

假设我们正在开发一个智能客服机器人，其中一个功能是回答用户关于某个城市未来某天的天气情况。用户可能以自然语言提问，例如"北京明天会下雨吗？"或"上海后天天气怎么样？"为了调用大模型获得准确回答，我们需要构建一个结构化的提示词，其中包含**城市**（city）和**日期**（date）这两个动态变量。

使用最简单的提示模板

```
template = """
你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：{city}
日期：{date}
"""
city = input('请输入要查询的城市：')
date = input('请输入要查询的日期：')
prompt = template.format(city=city,date=date)
```

2.1.2 问题

我们发现一个问题，如果用户第一次没有输入 date ， 我们第二次再往生成的 prompt 里面传参时，没什么效果

```
template = """
你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：{city}
日期：{date}
"""
# city = input('请输入要查询的城市：')
# date = input('请输入要查询的日期：')
city = '北京'
date = None
prompt = template.format(city=city,date=date)
print(prompt)
date = '明天'
final_prompt = prompt.format(city=city,date=date)
print(final_prompt)
```

输出结果：

```
你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：北京
日期：None


你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：北京
日期：None
```

原因解析：

Python 字符串的 `format` 方法是一次性的，执行后占位符 `{}` 就被替换成了具体值，字符串中不再保留任何占位符。因此第二次调用 `format` 时，因为字符串中没有 `{city}` 和 `{date}`，所以没有任何替换发生，只是原样返回了已经替换过的字符串（日期仍为 `None`）。

这正是原生字符串格式化的限制：**无法部分填充后保留剩余占位符。**一旦调用 `.format()`，所有占位符都会被一次性消耗掉。

不过难不到我们，直接拿template把两个变量输入就行了呗（狗头）

```python
final_prompt = template.format(city=city,date=date)
print(final_prompt)
```

2.1.3 新的问题

我们根据后台收到的数据，发现用户询问明天天气的比较多，或者用户不输入时间，产品经理要求我们给 date 一个默认参数。

当然，这也难不到我们，在代码开头的时候，提前设置一个date = "明天" 即可，用户输入新的date的时候直接覆盖掉就行。

这里要说明的是，在工程上，这种思想属于“战略上偷懒，战术上勤奋”。每次都老老实实把 `city` 和 `date` 两个变量都传给 `template.format()`，哪怕同一个城市要写十遍。代码看起来每一行都在“认真工作”，但整体重复、臃肿、易错。

解决方案：动态模板 `PromptTemplate`，通过模板语法，让提示词结构化，支持：

- 变量替换（如 `{question}`）
- 设置默认值
- 条件逻辑（可选参数）

实现代码：

```python
from langchain_core.prompts import PromptTemplate

# 定义模板（模板语法：用{}包裹变量名）
template = """
你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：{city}
日期：{date}
"""

```

```python
# 创建PromptTemplate实例
prompt = PromptTemplate(
    input_variables=["city", "date"],  # 声明模板需要的变量
    template=template
)

# 生成具体Prompt（传入变量值）
generated_prompt = prompt.format(city="北京", date="明天")
print(generated_prompt)
```

输出结果

```
你是一个天气查询助手。请根据用户提供的城市和日期，返回天气信息。
城市：北京
日期：明天
```

这里相当于使用 langchain 的集成模板，代替了前面我们自己写的Template

2.1.4 解决产品经理的问题

（1）封装默认值

```python
prompt = PromptTemplate(
    input_variables=["date", "city"],
    template=template,
    # 方法1：封装时预设默认值
    partial_variables={"date": "明天"}
)

# 即使不传city，也会使用默认值
print(prompt.format(city='北京'))
```

产品经理提要求了，你这样封装到提示模板里面，万一在别的地方，用户想问今天下午怎么办，用户想问昨天怎么办，给我多封装几个。

（2）部分传参

为了少写代码，也为了工程合理性，我们可以使用方法2：部分传参

```python
prompt = PromptTemplate(
    input_variables=["date", "city"],
    template=template
)
t1_prompt = prompt.partial(date = '明天')
t0_prompt = prompt.partial(date = '今天下午')
# 顺便封装一些城市的模板
beijing_prompt = prompt.partial(city = '北京')
```

2.2 角色模板

在大模型的对话体系中，每条消息都必须附带一个身份标签——即“角色”。这不是偶然的设计，而是一种必要的信息组织方式。主流大模型（如GPT-4、DeepSeek）的对话API使用三个核心角色来组织每一次对话：

2.2.1 LLM 里面的角色

1. System（系统角色）

System角色相当于对话的“导演”或“编剧”。它在对话开始时设定全局规则，定义整个对话的基调、约束和行为规范，但不直接参与交互。System角色告诉模型“你应该以什么身份来回答问题”“你可以做什么、不能做什么”。它可以是一个简短的句子，例如`You are a helpful AI assistant.`，也可以包含多行详细指令，例如“你是一个专业的技术支持专家，回答应当简洁且使用中文”。

System角色对于大语言模型来说是最顶层的控制指令，它为整个对话提供了一个稳定的“操作环境”。开发者可以通过修改System角色来“遥控”模型的行为——在复杂的应用场景中，System角色是开发者唯一可以全权控制的指令入口。

2. User（用户角色）

User角色就是“提问者”或“任务发起者”。它代表用户每一次输入的具体问题、指令或数据。User角色的内容是动态变化的，每一次用户的提问都用User消息封装。

3. Assistant（助手角色）

Assistant角色是大模型自身。它负责根据System设定的规则和User提出的问题，生成符合要求的回答。在多轮对话中，Assistant的过往回答会被保存在消息列表中，供模型在下一轮“回顾”以维持对话的连贯性。

这三大角色之间存在明确的排列约束：System必须出现在第一条且只能有一条，之后System/User/Assistant通常交替出现。

2.2.2 为什么要分角色?

角色的引入绝非形式主义，有以下三重必要性：

原因一：分离“规则”与“任务”

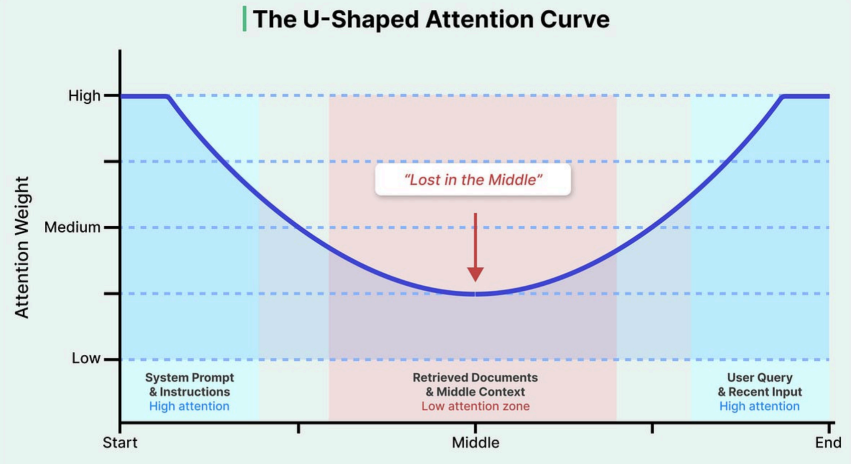
如果把规则和任务混在一起放入User消息中，大模型容易将规则误读为回答内容，出现错误响应。通过System承载规则、User承载任务，模型才能明确“谁是发令人”“谁是执行者”。

原因二：多轮对话的“记忆坐标系”

大模型本身没有真正的长期记忆，它靠的是消息列表中的历史上下文来“回忆”过往。如果不标记每句话是谁说的，模型无法判断一句“那明天呢”背后的“那”指的是什么——因为它分不清这句追问是自己的回复还是用户的新输入。角色标记为多轮对话提供了一种上下文关系的坐标体系。

原因三：开发者控制的第一入口

System角色是开发者调控AI行为最直接有效的渠道——你可以在System中一次性定义好模型的身份、输出格式、安全边界、回答风格等全局约束，而不需要在每次用户输入中反复声明。



2.2.3 对比测试角色扮演的效果

1.提取参数

```
# 把 system 的提示词也封装一下
    def invoke(self,prompt,max_tokens = 1000,temperature = 0.9,top_p = 0.95,
            stop = None,system_role = "你是一个有帮助的助手"):
        time1 = time.time()
        if not isinstance(prompt,list):
            # 如果prompt不是列表，则将其转换为列表
            prompt = [
                {"role":"system","content":system_role},
                {"role": "user", "content": prompt}
            ]
```

2.使用不同的 `system_role`

```
from root import *

system_role1 = "你是一个有帮助的助手"
system_role2 = "你是一位专业的美食评论家，擅长分析菜品的色、香、味、口感，并给出星级评分。"
system_role3 = "你是一名注册营养师，关注食物对健康的影响，擅长分析营养成分。"

prompt = "请评价一下北京特色食物-豆汁，生成字符数控制在300字以内"

print('对比效果如下：')
for role in [system_role1,system_role2,system_role3]:
    print('='*20)
    print(f'扮演的角色：{role}')
    print(llm.invoke(prompt,system_role=role))
```

2.2.3 `messages`解析

1.字段

`messages` 是一个**角色-内容对的有序列表**，每个元素包含两个关键字段：

字段	说明	示例值
`role`	消息发送者角色（决定行为逻辑）	`"system"`, `"user"`, `"assistant"`
`content`	消息正文（文本内容）	`"你是一个数学助手"`, `"1+1等于多少？"`

2. `langchain_core.messages`

`langchain_core.messages` 定义了对话系统的**标准化数据结构**，核心消息类型包括：

底层 API 角色	LangChain 的封装类	作用说明
`system`	`SystemMessage`	用于设定模型的**系统指令**，例如定义它的角色、行为准则或提供背景信息。
`user`	`HumanMessage`	代表**用户的输入**，即人类提出的问题或指令。
`assistant`	`AIMessage`	代表**模型的回复**。在多轮对话中，你需要将模型的回复（作为 `AIMessage`）传回，以维持上下文。

这些类统一了消息格式（`type` 和 `content` 字段），确保与LLM模型（如 `ChatOpenAI`）无缝兼容。

2.2.4 构建方法

1.直接构造（显式控制）

```python
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage(content="你是一个专业的数学题解答助手。"),   # 系统规则
    HumanMessage(content="请计算3+5的结果"),                  # 用户提问
    AIMessage(content="根据计算，答案是：8")                  # 助手回答（可选）
]
print(messages)
```

2.通过模板生成（推荐）

```python
from langchain_core.prompts.chat import (
    ChatPromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate
)
```

```python

# 定义模板
system_template = SystemMessagePromptTemplate.from_template(
    template="你是一个专业的数学题解答助手。"
)
user_template = HumanMessagePromptTemplate.from_template(
    template="{user_question}"
)

chat_template = ChatPromptTemplate.from_messages([
    system_template,
    user_template
])

# 填充变量生成messages
messages = chat_template.format_messages(user_question="请计算3+5的结果")
print(messages)
```

2.3 多轮对话技术

2.3.1 背景介绍

在前面的学习中，我们已经能够通过API调用大模型，每次向模型发送一个问题，获得一个回答。但这种调用方式存在一个明显的局限：**模型不记得你之前说过什么。**

你问“北京明天天气怎么样”，模型回答了；接着你问“那后天呢”，模型无法理解“那”指的是什么，因为它看不到上一轮对话。而市面上所有的对话产品（ChatGPT、DeepSeek网页版、Kimi等）都具备“记忆”能力——它们能够基于完整的对话历史给出连贯的回答。

本项目将实现一个**可连续对话、可自定义角色、具备完整记忆能力的命令行聊天机器人。**完成后，你将拥有一个不依赖任何网页产品、完全由自己控制的AI对话程序。

2.涉及的技术点

- **消息角色**：使用LangChain的`SystemMessage`、`HumanMessage`、`AIMessage`区分系统设定、用户输入和模型回复。
- **对话历史管理**：用一个Python列表按时间顺序存储所有消息，每次调用API时都将整个列表传入，让模型“看到”完整的上下文。
- **角色扮演**：通过`SystemMessage`在对话开始前设定模型的身份和行为风格，该设定会在整个对话过程中持续生效。
- **交互式循环**：使用`while True`构建连续对话界面，支持退出命令。
- **环境变量配置**：API密钥通过环境变量读取，避免硬编码。

2.3.2 核心技术讲解

1.实现记忆原理

大模型本质上是一个**无状态的函数**：每次调用时，它只根据当前输入的内容（提示词 + 用户消息）来预测输出。它不会主动记住你上一次问了什么，也不会记录自己上一次回答了什么。每一次调用都是"独立的"、"全新的一次"。

虽然模型没有记忆，但它的设计允许你**把之前的对话历史作为输入的一部分**一起传给它。也就是说，每一次调用时，你可以把前几轮的所有对话（用户说了什么、模型回了什么）按顺序写成一个列表，然后放在当前问题前面，一起发给模型。

2.具体操作

我们维护一个固定长度的窗口，作为上下文记忆，这个记忆本质上就是对话流水账

在代码中，我们用一个 Python 列表 `messages` 来存储每一轮的消息。每轮对话：

1. 用户说的话 → 追加为 `HumanMessage`
2. 模型的回复 → 追加为 `AIMessage`

下次调用时，把整个 `messages` 列表传回给模型。模型看到的就是"之前所有的对话 + 最新的用户问题"。

这就是"记忆"的全部秘密：不是模型记住了，而是你每次都把过去的聊天记录重新给它看一遍。

3.为什么窗口

模型能接受的输入长度是有限的（比如 4K、8K、128K token）。如果你把无限长的历史都塞进去，迟早会超过这个上限。因此，实际上不能保留全部历史，只能保留**最近 N 轮对话**，像一个滑动窗口一样向前滚动。

"列表窗口"就是指：我们只维护最近的若干条消息（比如最近 10 轮），超出长度的旧消息直接丢弃或压缩。这样既保证了模型能看到必要的上下文，又不会超过长度限制。

另外，随着记忆越来越多，如果把全部对话记录塞入，也会导致 token 消耗激增。

同时，越久远的对话消息，对当前聊天内容影响越小，保持一定长度的上下文窗口，可以保证模型带有一定记忆，又可以忽略过去久远的消息。

4.映射转换

在langchain的提示模板里面，并不能把messages直接放入到OpenAI这种原生态模型里面，我们需要做一个映射转换。

```python
# 可以定义在 root 里面
def convert(messages):
    """批量转换任意消息类型为模型格式"""
    mapping = {
        "human": "user",
        "ai": "assistant",
        "system": "system",
    }
    return [
        {
            "role": mapping[msg.type],
            "content": msg.content
        }
        for msg in messages
```

```
    ]
```

这里只需要在invoke里面，把原本的messages，转换成我们对应的对话记录即可。

```
        if not isinstance(prompt,list):
            # 如果prompt不是列表，则将其转换为列表
            prompt = [
                {"role":"system","content":system_role},
                {"role": "user", "content": prompt}
            ]
        prompt = convert(prompt)    #  <--- 添加这句
        response = self.client.chat.completions.create(
            # ...
```

2.3.3 具体实现

1.启动大模型开始对话

```
# 1.原神启动
system_template = SystemMessagePromptTemplate.from_template(
    template = '''
你是我的一个带有记忆的聊天机器人，可以根据我们的对话保存历史记忆
''')
user_template = HumanMessagePromptTemplate.from_template(
    template="{user_question}"
)
chat_template = ChatPromptTemplate.from_messages(
    [system_template,
    user_template]
)
messages = chat_template.format_messages(user_question = input())
print(messages)
print(convert(messages))
```

2.设定循环主体

```
def main(memory = 6,quiting = '0'):
    print("AI 机器人为您服务")
    while True:
        query = input("请输入你的聊天内容: ")
        if query == '0':
            break
        if len(messages) >= 6:
            messages.pop(0)

        user_prompt = HumanMessage(content=query)
        messages.append(user_prompt)
        ai_answer = llm.invoke(messages)
        if len(messages) >= 6:
            messages.pop(0)
```

```
        ai_prompt = AIMessage(content=ai_answer)
        messages.append(ai_prompt)
        print(f'用户输入：{query}')
        print(f'AI回复：{ai_answer}')
main()
```

生成结果：

```
AI 机器人为您服务
请输入你的聊天内容：你是华人牌手机傻妞，现在开机
推理耗时：1.07秒
用户输入：你是华人牌手机傻妞，现在开机
AI回复：傻妞开机成功！主人，欢迎使用华人牌手机傻妞，我是您的贴心智能助手，请问有什么可以为您服务的？
请输入你的聊天内容：你有什么功能
推理耗时：1.39秒
用户输入：你有什么功能
AI回复：主人，傻妞拥有丰富的功能呢！我可以帮您拨打电话、发送短信、查询天气、播放音乐、设置闹钟、记录备忘录、导航路线，还可以随时陪您聊天解闷。另外，如果您需要，我还能学习您的喜好，提供更个性化的服务哦！主人，有什么需要我做的吗？
请输入你的聊天内容：你猜猜你为什么叫傻妞
```

3.提示工程（经典）

3.1 概述

3.1.1 历史发展

（1）什么是提示词工程

提示词工程（Prompt Engineering）是指系统性地设计、优化和迭代输入提示词，以引导大语言模型（LLM）产生符合预期、稳定且高质量的输出。其本质是通过精心构造上下文输入，在不修改模型参数的情况下显著提升模型的推理能力、输出可控性和任务适应性。

（2）关键里程碑

- 2017年：Transformer架构的提出为长上下文理解奠定基础，使提示词能够有效承载复杂指令。
- 2020-2021年：GPT-3引入上下文学习（In-Context Learning），证明仅通过提示词即可让模型快速适应新任务，无需微调。
- 2022年：Chain-of-Thought（CoT，思维链）技术由Google研究者提出，在GSM8K数学基准测试中将准确率从17.7%提升至78.7%，标志着提示词工程从经验技巧走向系统方法。
- 2022-2023年：Self-Consistency（自洽性）、Tree-of-Thoughts（思维树）、ReAct等高级框架相继出现，进一步提升复杂推理、规划和决策任务的表现。
- 2023年ChatGPT广泛应用后：提示词工程从学术研究转变为行业核心技能，企业开始设立专门的提示词工程师岗位。
- 2025年后：提示词工程演进为上下文工程（Context Engineering），与Agent、RAG和工具调用深度融合，强调自动化优化与安全性。

3.1.2 定义与核心目标

提示词工程是通过结构化设计提示词，使大语言模型能够可靠地完成复杂任务的技术体系。其核心不再是简单提问，而是构建包含角色设定、示例指导、推理步骤和输出约束的完整输入框架。

核心目标包括：

1. 提升模型推理的透明度和准确性；
2. 降低输出幻觉和不一致性；
3. 实现输出格式的可控性，便于程序集成；
4. 增强模型在多步任务中的结构化思考能力。

3.1.3 解决的主要问题

基础提示词在实际应用中存在以下主要局限性，提示词工程正是针对这些问题而发展：

1. 推理跳跃或错误：模型直接输出答案，中间逻辑不透明，易导致数学、逻辑等复杂任务失败。
2. 幻觉与不一致：同一问题多次查询结果差异较大，或生成不存在的信息。
3. 复杂任务失效：涉及多步规划、对比分析或创造性思考时，模型表现不稳定。
4. 输出不可控：格式混乱，无法直接用于下游程序处理。
5. 安全性风险：易受Prompt注入、奶奶漏洞等攻击绕过防护机制。

通过高阶提示技巧，可显著降低上述问题，使模型输出更具可解释性、可验证性和可靠性。

3.1.4 当前发展情况（截至2026年）

2026年的提示词工程已从单纯的“Prompt Engineering”演变为“Context Engineering”。前沿模型（如OpenAI o1系列、Claude 3.5/4、Gemini 2）已内置部分推理能力，显式提示词的作用从必要条件转变为增强信号。

主要发展趋势包括：

1. 与 Agent、RAG 和 Tool Calling 的深度集成，形成端到端解决方案；
2. 自动化提示优化（Meta-Prompting）和动态上下文管理；
3. 强调可解释性、安全性和鲁棒性；
4. 传统“Prompt Engineer”岗位数量减少，但提示词工程思维已成为所有AI从业者的必备核心能力。

一句话总结：高阶提示词工程以最低成本最大化利用现有大模型的潜力，将模型从“聪明助手”升级为“可信搭档”。

3.1.5 提示词技巧分类

1.基础技巧（Foundational Prompting）

特点：通过简洁直接的指令设计，快速帮助模型理解任务意图并输出合理、稳定的结果，适合大多数日常场景，是提示工程的入门基础。

方法：

- **明确任务说明**：用精确、无歧义的语言直接定义任务目标、输出格式和约束条件，确保模型一次性抓住核心需求。
- **角色设定**：为模型赋予特定身份或专业背景，引导其以对应视角、语气和知识深度进行回应。
- **提供示例（Few-shot/One-shot）**：在提示中给出少量输入-输出对照样本，帮助模型快速学习期望的格

式、风格和逻辑模式。

- **输入规范化**：对输入数据的格式、长度、语言、结构等进行明确限制，减少模型理解偏差。

应用场景：

- 简单问答、文本分类、摘要提炼、翻译、信息提取等结构化基础任务。

2.进阶技巧（Intermediate Prompting）

特点：在基础理解之上，重点强化模型的推理逻辑、透明度和输出一致性，使答案具备可解释性和更强的可靠性。

方法：

- **思维链（Chain of Thought, CoT）**：引导模型逐步拆解问题、展示中间推理步骤，从而提升复杂问题的解决准确率。
- **生成知识提示**：先让模型生成相关背景知识、结构化框架或对比分析，再基于这些知识完成最终任务。
- **自洽性（Self-Consistency, SC）**：通过多次生成不同推理路径，再选取最一致或最高频的结果作为最终答案，提高输出稳定性。
- **条件生成**：在提示中设定特定约束条件、上下文范围或优先级规则，控制生成方向和深度。

应用场景：

- 复杂数学或逻辑推理、科学原理解释、多维度决策分析、需要透明推理过程的任务。

3.高级技巧（Advanced Prompting）

特点：针对高复杂度、高风险或需要创新与可靠性的场景，进一步提升模型的探索能力、决策质量和系统安全性，实现从单次生成向智能协作的跃升。

方法：

- **思维树（Tree of Thought, ToT）**：采用分支式探索，同时生成多条推理路径，根据中间评估动态剪枝或扩展最优路径。
- **多目标优化提示**：在提示中明确多个评价维度及权重，实现多目标平衡决策，避免单一维度偏差。
- **防范 Prompt 攻击**：通过上下文验证、指令强化、输出过滤等机制，防止提示注入、越狱或恶意操纵。
- **自适应交互提示**：根据模型中间输出实时调整后续提示，实现动态反馈循环与迭代优化。

应用场景：

- 高风险领域决策（医疗、法律、金融等）、多步骤复杂推理与创作、系统级多模型协作、需要高可靠性和创新性的长期任务。

3.2 基础技巧：Few-Shot Prompting

Few-Shot Prompting（少样本提示）是提示词工程中最常用、最基础的高阶技巧之一。它通过在提示词中提供少量（通常为 2~5 个）输入-输出示例，使模型在无需任何参数更新或额外训练的情况下，快速学习并泛化到新任务。

3.2.1 什么是 Few-Shot Prompting

Few-Shot Prompting 的核心思想源于**上下文学习（In-Context Learning）**：模型在接收到提示词中的示例后，能够根据这些示例的模式、格式和规则，推断出用户期望的输出方式。

- **Zero-Shot Prompting（零样本提示）**：仅给出任务描述，不提供任何示例。模型完全依靠预训练知识完成任务。
- **One-Shot Prompting（单样本提示）**：仅提供 1 个输入-输出示例。
- **Few-Shot Prompting（少样本提示）**：提供 2~5 个（或更多）高质量示例，通常效果优于 Zero-Shot 和 One-Shot。

Few-Shot 并非让模型"记住"示例，而是利用模型在预训练阶段已掌握的模式识别能力，通过上下文中的示例"激活"相应的能力。

3.2.2 核心原理

1. **模式匹配**：模型会自动识别示例中的输入-输出映射关系，并将其应用到新的输入上。
2. **格式对齐**：示例能明确告诉模型期望的输出格式（如 JSON、表格、特定语气）。
3. **任务泛化**：通过少量示例，模型可以从具体案例推导出通用规则，避免直接依赖内置知识导致的偏差。

与微调（Fine-Tuning）相比，Few-Shot Prompting 的优势在于：

- 无需准备大量标注数据；
- 无需消耗 GPU 资源进行训练；
- 修改示例即可快速迭代，成本极低。

3.2.3 示例：提示词实现感情分析

```
from langchain_core.prompts import PromptTemplate

# 1. 定义提示词模板（已按标准结构划分）
prompt_template = PromptTemplate(
    input_variables=["input"],
    template="""角色（Role）：你是一个专业的电影评论情感分类专家。

任务（Task）：根据用户输入进行情感分类。

约束（Constraint）：
- 分类只能是“积极”、“消极”或“中性”三类之一
- 必须严格返回 JSON 格式，不要添加任何额外文字或解释

示例（Few-shot）：
输入：这部电影太棒了，剧情精彩，演员演技一流！
输出：{{"category": "积极", "confidence": 0.95, "reason": "使用了“太棒了”“精彩”“一流”等正面词汇"}}

输入：剧情拖沓，特效很假，完全浪费时间。
输出：{{"category": "消极", "confidence": 0.92, "reason": "提到“拖沓”“很假”“浪费时间”等负面描述"}}

```

```
输入：故事还行，演员表演一般，整体中规中矩。
输出：{{"category": "中性", "confidence": 0.80, "reason": "没有明显褒贬词语，评价平淡"}}

内容（Context）：
输入：{input}

输出格式（Output Format）：严格按照 JSON 格式输出，包含 category、confidence、reason 三个字段。"""
)

# 2. 准备用户输入
user_input = "这是一部让人眼前一亮的科幻大片，特效震撼，但结尾有点仓促。"

# 3. 生成最终提示词
final_prompt = prompt_template.format(input=user_input)

print("=== 发送给模型的完整提示词 ===\n")
print(final_prompt)
print("\n" + "="*60 + "\n")

# 4. 调用模型
result = llm.invoke(final_prompt)
print("=== 模型返回结果 ===\n")
print(result)
```

3.3 进阶技巧1：CoT

3.3.1 概述

CoT（Chain of Thought，思维链）是提示工程中最为经典且实用的进阶技巧之一。它通过在提示中明确要求模型“一步一步思考”，让大语言模型不再直接跳到最终答案，而是先将复杂问题拆解成一系列中间推理步骤，从而模拟人类逐步思考的完整过程。这种方式从根本上改变了模型的生成逻辑，把原本“黑箱式”的端到端输出，转变为可追踪、可解释的多步推理链。

CoT的最大优势在于显著提升了模型在复杂任务上的表现。它特别适合需要严谨逻辑的场景，例如数学题求解、逻辑推理、多跳问答或科学原理分析。

通过显式输出每一步中间推理，模型的错误率大幅降低，同时输出的可解释性也大大增强——用户或开发者可以轻松查看模型到底是怎么一步步得出结论的，这在调试和审核时非常实用。更重要的是，CoT的实现成本极低，只需修改提示词即可生效，无需额外训练或微调，无论是API调用还是本地部署都能立即看到效果。

3.3.2 具体示例

为了更直观地展示CoT的实际工作方式，我们以经典的“鸡兔同笼”数学题作为场景。这个问题看似简单，却能很好地体现CoT如何将模型从“直接猜答案”转变为“一步一步严谨推理”。

题目： "鸡兔同笼，共有35个头、94条腿。已知鸡有2条腿，兔有4条腿。请问鸡和兔各有多少只？"

1.传统提示（无思维链）

```
问题：鸡兔同笼，共35只，腿共94条，鸡和兔子各多少只？
答案：
```

潜在问题： 如果不使用CoT，模型有时可能直接给出答案却无法解释，甚至在类似变形题中出错。

2.思维链提示（分步引导）

```
问题：鸡兔同笼，共35只，腿共94条，鸡和兔子各多少只？

请按照以下步骤思考：
1. 假设所有动物都是鸡，计算总腿数。
2. 比较实际腿数与假设腿数的差异。
3. 分析差异来源（每只兔子比鸡多2条腿）。
4. 计算兔子的数量。
5. 最终得出鸡和兔子的数量。

逐步推理：
```

3.模型生成的思维链过程

```
1. 假设所有35只都是鸡，每只鸡2条腿，总腿数应为 35×2=70 条。
2. 实际腿数为94条，比假设多出 94-70=24 条。
3. 每只兔子比鸡多2条腿（4-2=2），因此多出的24条腿对应 24÷2=12 只兔子。
4. 兔子数量为12只，鸡的数量为 35-12=23 只。
5. 验证：23只鸡有46条腿，12只兔有48条腿，总计46+48=94条，符合条件。

答案：鸡23只，兔12只。
```

3.3.3 具体实现

核心思想：让模型**一步一步思考**（step by step），而不是直接给出答案，从而显著提升复杂推理的准确率。

适用场景：数学题、逻辑推理、因果分析等。

```python
from langchain_core.prompts import PromptTemplate

# 实验题目
question = "鸡兔同笼，共35只，腿共94条，鸡和兔子各多少只？"

# CoT 提示词
cot_prompt = PromptTemplate.from_template(
    """任务（Task）：一步一步思考并解决数学问题。

约束（Constraint）：
- 请详细解释整个思考过程
- 最后只在最后一行给出最终答案

```

```
内容（Context）：
问题：{question}"""
)

prompt = cot_prompt.format(question=question)

print("=== CoT 提示词 ===\n", prompt)

# 调用模型
llm_output = llm.invoke(prompt)
print("\n模型输出（思维链）：\n", llm_output)
```

3.3.4 当代大模型

当代大语言模型在处理像“鸡兔同笼”这样的经典数学题时，即使提示词里完全不提“一步一步思考”，也极少出错。这不是因为CoT技巧突然失效了，而是模型的底层能力发生了质的飞跃，把原本需要显式引导的推理过程，变成了模型“内置”的默认行为。原因有以下几点：

1.训练方式的革命性升级

- 早期模型（如GPT-3时代）主要靠海量文本“模仿”人类语言，遇到需要多步推导的问题时，容易直接跳结论、漏掉中间环节，准确率很低。因此研究者才发明CoT，通过提示强行把推理步骤“逼”出来。
- 但从GPT-4开始，训练数据中大量注入了带完整推理链的合成数据：包括数学教材、编程题解题过程、科学论文推导、甚至用早期模型自己生成的CoT数据再进行自我迭代（类似Self-Taught Reasoner或STaR方法）。

模型不再只是“记住答案”，而是真正学会了“如何思考”的模式。

2.后训练（post-training）阶段的强化

通过RLHF/RLAIF、过程监督（Process Supervision）和Outcome Supervision，模型在每一步中间推理上都获得了奖励信号，而不是只看最终答案是否正确。这让模型在生成答案时，内部就已经自动完成了隐式思维链——即使你看不见，它在Transformer的隐藏层里已经把“设未知数→列方程→化简→验证”这些步骤走了一遍，再一次性输出正确结果。

3.现代模型的架构优化

更长的上下文窗口（128k甚至1M+ tokens）、混合专家（MoE）结构、更高效的注意力机制，都让模型能同时“记住”更多中间变量，而不会像早期模型那样“忘掉”前面的条件。所以在鸡兔同笼这类中等难度的封闭问题上，直接问“鸡兔各多少只？”就能得到 23 和 12 的正确答案，几乎不需要额外提示。

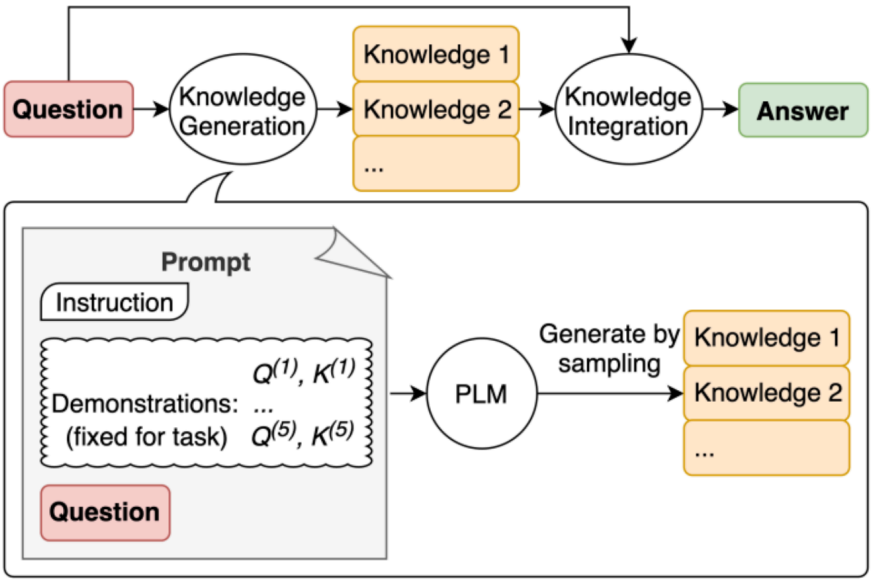
当然，这并不意味着CoT已经过时。对于极复杂、多跳、开放式或高风险任务（比如全新数学竞赛题、跨领域法律分析、需要严格可审计的医疗决策），显式CoT仍然能进一步降低错误率、提高一致性，并提供可解释的推理路径。只是对绝大多数日常问题，当代模型已经把“思维链”从提示词层面，升级成了模型的“本能”。

3.4 进阶技巧2：生成知识提示

3.4.1 概述

生成知识提示（Generate Knowledge Prompting）是提示工程中非常经典且实用的进阶技巧之一。它通过在提示中明确要求模型先“生成相关背景知识、结构化框架或对比分析”，再让模型基于这些自生成的知识完成最终任务，让大语言模型不再直接跳到答案，而是先主动构建一个完整、相关的知识基础，从而模拟人类“先整理知识、再作答”的完整过程。

这种方式从根本上改变了模型的生成逻辑，把原本可能依赖预训练记忆或容易产生幻觉的端到端输出，转变为可追踪、可解释的两阶段生成过程。



3.4.2 两阶段生成

1.第一阶段：知识生成（Knowledge Generation Phase）

这一阶段的核心任务是让模型**主动产出与任务相关的背景知识、结构化框架或分析内容。**模型在此阶段专注于“整理知识”，不直接回答原始问题，而是为后续决策提供坚实的基础。通过这一步，模型可以有效减少幻觉、补充可能遗漏的信息，并形成清晰的知识框架。

第一阶段常见的**三种生成方式**如下：

1. **知识生成（Direct Knowledge Generation）**：直接、简洁地输出事实列表、结构化框架或关键信息，无需额外推理步骤，适用于需要快速补充上下文的场景。
2. **推理链生成（CoT-based Knowledge Generation）**：结合思维链（CoT），引导模型一步一步推理、拆解并生成知识框架，适用于需要逻辑严谨、多步推导的复杂知识构建。
3. **对比分析生成（Comparative Analysis Generation）**：要求模型生成对比表格、优缺点分析或多维度框架（如SWOT），适用于决策类、评估类或多方案对比任务，能让知识更具结构化和平衡性。

2.第二阶段：知识整合（Knowledge Integration Phase）

这一阶段的任务是**基于第一阶段生成的全部知识**，完成原始问题的最终分析、决策、建议或答案输出。模型在此阶段将第一阶段的知识作为可靠上下文，进行综合推理和应用，最终给出清晰、可追溯的结果。

通过这种两阶段设计，生成知识提示不仅提高了任务完成质量，还让整个过程变得透明可审计，便于开发者或用户进行检查和迭代。

3.4.3 示例

1.知识生成（Direct Knowledge Generation）

目标：让模型直接、简洁地输出详细的背景知识、事实列表或结构化框架，无需额外推理步骤。

示例：

```
提示：请直接生成2026年电动汽车与传统燃油车的背景知识框架，涵盖购置成本、运营成本、环保性能、技术成熟度、政策支持、市场趋势、便利性等维度。
```

```
模型输出：
2026年电动汽车与传统燃油车背景知识框架：
• 购置成本：电动汽车18-25万，燃油车15-22万，电动车可享最高2万补贴。
• 运营成本：电动汽车每公里0.15元，燃油车约0.55元，5年养车成本电动车低约40%。
• 环保性能：电动汽车全生命周期碳排放低60-70%，燃油车受碳税和限行影响较大。
• 技术成熟度：电动汽车续航600-800公里，充电网络覆盖率95%以上。
• 政策支持：2026年多城市继续限购燃油车，电动汽车免购置税+停车优惠。
• 市场趋势：电动汽车市场份额预计达45%，二手残值率提升。
• 便利性：日常通勤电动汽车更友好，长途旅行燃油车仍有优势。
```

2.推理链生成（CoT-based Knowledge Generation）

目标：强制模型通过完整推理链条生成知识，即使后续答案有误也能追溯根源。 **示例**：

```
提示：请逐步推导2026年普通三口之家选择电动汽车还是燃油车的知识框架，并标注每一步的依据。

模型输出：
1. 分析家庭用车场景：三口之家日常通勤为主，偶尔长途。
2. 成本维度推导：电动汽车初始购置略高，但5年总拥有成本因电费低而显著下降。
3. 环保与政策维度推导：2026年碳税和限行政策趋严，电动汽车政策红利更大。
4. 便利性维度推导：城市充电桩已高度覆盖，长途场景燃油车仍具优势。
5. 综合知识框架形成：电动汽车在城市家庭场景下综合优势更明显。
```

3.对比分析生成（Comparative Analysis Generation）

目标：让模型自主生成多维度对比表格或框架，辅助后续决策。

示例：

```
提示：请用对比表格的方式生成2026年电动汽车与传统燃油车的知识框架，从环保性、成本、续航/便利性三个维度展开。

模型输出：
| 维度          | 电动汽车                          | 燃油车                          |
|-------------|-------------------------------|-----------------------------|
| 环保性         | 零排放，全生命周期碳排放低60-70% | 尾气排放高，受碳税限制         |
| 使用成本      | 电费低，维护简单，5年省40%     | 油费高，保养费用较高          |
| 续航/便利性| 城市友好，长途需规划充电        | 加油便捷，长途无焦虑          |
```

3.4.4 具体实现

核心思想：**先让模型生成相关知识/事实**，然后用这些知识来回答问题。

相当于“先查资料，再回答”，能大幅减少幻觉，提高准确性。

```python
from langchain_core.prompts import PromptTemplate

# ==================== 实验主题 ====================
topic = "2026年普通三口之家选择电动汽车还是传统燃油车"

# ==================== 第一步：生成综合知识框架 ====================
knowledge_prompt = PromptTemplate.from_template(
    """任务（Task）：生成电动汽车与传统燃油车的背景知识框架。

约束（Constraint）：涵盖购置成本、运营成本、环保性能、技术成熟度、政策支持、市场趋势、便利性等维度。

内容（Context）：主题：{topic}"""
)

prompt1 = knowledge_prompt.format(topic=topic)
print("=== 第一步：生成综合知识框架 ===\n")
print(prompt1)

knowledge = llm.invoke(prompt1)
print("\n生成的知识框架：\n", knowledge)


# ==================== 第二步：逐步推导决策框架 ====================
reasoning_prompt = PromptTemplate.from_template(
    """任务（Task）：逐步推导三口之家选择电动汽车还是燃油车的知识框架。

约束（Constraint）：每一步都要标注依据。

内容（Context）：主题：{topic}

输出格式（Output Format）：请按以下格式输出：
1. ...
2. ...
3. ..."""
)
```

```python

prompt2 = reasoning_prompt.format(topic=topic)
print("\n=== 第二步：逐步推导决策框架 ===\n")
print(prompt2)

reasoning_result = llm.invoke(prompt2)
print("\n逐步推导结果：\n", reasoning_result)


# ==================== 第三步：生成对比表格 ====================
table_prompt = PromptTemplate.from_template(
    """任务（Task）：用对比表格的方式生成电动汽车与传统燃油车的知识框架。

约束（Constraint）：重点从环保性、成本、续航/便利性三个维度展开。

内容（Context）：主题：{topic}

输出格式（Output Format）：请直接输出 Markdown 表格，不要额外解释。"""
)

prompt3 = table_prompt.format(topic=topic)
print("\n=== 第三步：生成对比表格 ===\n")
print(prompt3)

table_result = llm.invoke(prompt3)
print("\n对比表格：\n", table_result)
```

3.4.5 设计意图

在本节中，针对“2026年三口之家电动汽车与燃油车选择”这一主题，设计了三个不同的 PromptTemplate。其目的是通过对比不同提示策略，展示提示词设计对模型输出结构和质量的影响。

（1）设计三个提示的原因

同一个任务可以采用多种不同的提示方式，而提示策略的差异会直接影响模型输出的广度、逻辑性和结构化程度。通过呈现三种风格不同的提示设计，可以清晰地展示提示词工程中策略选择的重要性。

（2）三个提示步骤的特点与作用

步骤	提示策略	主要特点	适用场景
第一步	直接生成综合知识框架	覆盖多个维度，信息较为全面	需要快速构建背景知识体系时
第二步	逐步推导 + 标注依据	强调推理过程和逻辑链条	需要较强可解释性和思考过程时
第三步	表格形式输出	强制结构化输出，便于对比	需要清晰、规范的对比结果时

（3）整体设计思路

本示例通过以下方式体现提示词工程的核心思路：

- **策略多样性**：展示同一任务可以采用不同提示方式，而非只有单一标准写法。

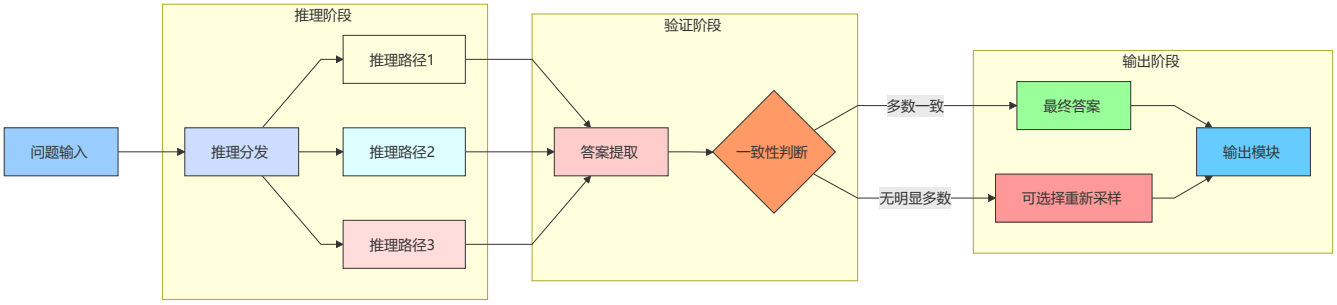
- **输出控制**：通过不同指令分别实现知识广度、推理深度和格式结构化三种不同目标。
- **方法对比**：让不同提示策略产生的输出结果形成鲜明对比，便于分析提示词设计的影响。
- **进阶铺垫**：为后续学习提示链（Prompt Chaining）、结构化输出控制等内容提供基础认知。

3.5 进阶提示技巧3：自洽性（Self-Consistency）

3.5.1 概述

自洽性（Self-Consistency, SC）是提示工程中重要的进阶技巧。它通过**为同一个问题生成多条独立的推理路径**，然后对比这些路径的结果，选择最一致（出现次数最多）的答案作为最终输出，从而大幅提高模型在复杂任务上的稳定性和准确率。

SC的核心思想是“让模型自己投票”：一条路径可能出错，但多条路径同时出错同一地方的概率很低。SC通常与CoT结合使用（即每条路径都走CoT），实现成本低，只需在提示中要求模型多次采样即可生效。



自洽性（Self-Consistency, SC）**本质上就是提示工程领域的“集成学习”。**它和机器学习中的随机森林（Random Forest）思想几乎一模一样：

- 随机森林：用多个决策树（每棵树随机采样数据和特征）分别预测，最后**多数投票**得到最终结果。
- 自洽性（SC）：用同一个模型生成**多条独立的推理路径**（每条路径像一棵“决策树”），最后**多数投票**选出最一致的答案。

SC 就是把“单模型”变成了“集成模型”，极大降低了单次推理出错的概率。

3.5.2 简单的示例：复杂数学逻辑题

（1）设计思路

问题类型：用不同方法计算同一问题（公式 vs 分解 vs 模拟）

示例：有 5 个红球和 5 个蓝球，随机抽取 3 个球。求至少有 2 个红球的概率。

自洽性点：

- 路径1：直接组合公式计算
- 路径2：分步计算「两红一蓝」+「三红」
- 路径3：用模拟抽样（Monte Carlo）估算

对比三条路径，确保结果一致。

（2）实际效果

```python
from langchain_core.prompts import PromptTemplate

question = """有 5 个红球和 5 个蓝球，随机抽取 3 个球。
求至少有 2 个红球的概率。"""

self_consistency_prompt = PromptTemplate.from_template(
    """角色（Role）：你是一个严谨的数学问题求解者。

任务（Task）：使用至少三种不同的方法独立计算概率问题，并对结果进行对比验证。

内容（Context）：问题：{question}

约束（Constraint）：
1. 使用三种不同的解决路径分别计算（例如：直接组合公式、分步枚举计算、蒙特卡洛模拟估算等）。
2. 每种方法都要详细展示计算过程和中间结果。
3. 最后对比三种方法的计算结果，如果结果一致则给出最终答案；如果存在差异，请分析原因并说明最可靠的结果。

输出格式（Output Format）：
【方法一：直接组合公式】
计算过程：
最终结果：

【方法二：分步枚举计算】
计算过程：
最终结果：

【方法三：蒙特卡洛模拟估算】
计算过程：
最终结果：

【结果对比与结论】
三种方法的结果是否一致？
最终答案："""
)

prompt = self_consistency_prompt.format(question=question)
print(prompt)
```

3.5.3 简单示例：算数推理 + 条件限制题

（1）设计思路

问题类型：对条件进行多角度检查

示例：一个公司有 A、B、C 三个部门，员工总数 30 人，A 比 B 多 2 人，C 是 B 的两倍。求每个部门人数。

自洽性点：

- 路径1：代数方程求解
- 路径2：穷举法 / 枚举可能性
- 路径3：逻辑检查总人数是否符合

多路径验证减少错误。

（2）具体示例

```
from langchain_core.prompts import PromptTemplate

question = """一个公司有 A、B、C 三个部门，员工总数 30 人。
A 比 B 多 2 人，C 是 B 的两倍。求每个部门的人数。"""

self_consistency_prompt = PromptTemplate.from_template(
    """角色（Role）：你是一个严谨的数学问题求解者。

任务（Task）：使用至少三种不同的方法独立求解问题，并进行一致性验证。

内容（Context）：问题：{question}

约束（Constraint）：
1. 使用三种不同的解决路径分别计算（例如：代数方程求解、穷举/枚举法、逻辑总和验证等）。
2. 每种方法都要详细展示计算步骤和中间结果。
3. 最后对比三种方法的计算结果，如果结果一致则给出最终答案；如果存在差异，请分析原因。

输出格式（Output Format）：
【方法一：代数方程求解】
计算过程：
最终结果：

【方法二：穷举 / 枚举法】
计算过程：
最终结果：

【方法三：逻辑总和验证】
计算过程：
最终结果：

【结果对比与最终答案】
三种方法的结果是否一致？
最终答案："""
)

prompt = self_consistency_prompt.format(question=question)
print(prompt)
```

3.5.4 小结

（1）核心思想

Self-Consistency 的核心思想是：**通过生成多条不同的推理路径，并选择出现次数最多的答案作为最终结果，来提高模型回答的可靠性。**

它认为：如果模型通过不同方式思考同一个问题，最终得出了相同的结果，那么这个答案出错的概率会显著降低。

简单来说，就是**“多路径思考 + 结果一致性验证”**。

（2）主要效果

- **提升准确率**：相比普通 Chain-of-Thought（CoT），在数学、逻辑、常识推理等任务上通常能带来明显的准确率提升。
- **减少单路径错误**：普通 CoT 只走一条推理链，容易因某一步出错而导致最终答案错误；Self-Consistency 通过多条路径验证，降低了这种风险。
- **增强鲁棒性**：对同一问题的一致性答案，更容易被认为是可靠的。

（3）代价与局限

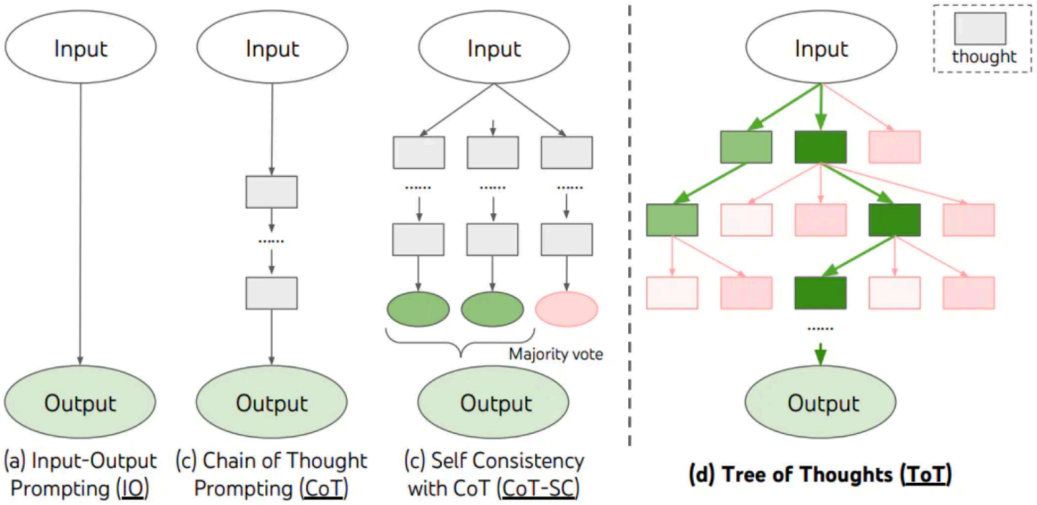
- **成本更高**：需要生成多条推理路径，token 消耗和推理时间会明显增加。
- **不是万能的**：如果所有路径都系统性出错，一致性反而可能强化错误答案。

3.6 高级提示技巧1：思维树

3.6.1 概述

思维树（Tree of Thought, ToT）是提示工程中最高阶的推理技巧之一。它让大语言模型不再沿着一条线性路径思考，而是**像一棵树一样同时生成多个思考分支**，通过“探索—评估—剪枝—扩展”的过程，主动搜索并收敛到最优解。

与传统提示方法最大的不同在于：ToT把模型的思考过程从“单线叙述”升级为**结构化的搜索过程。**模型会同时考虑多种可能性，对每条路径进行实时评估，主动放弃低价值分支，并对高潜力分支进行更深入的挖掘，最终从众多路径中选出最佳方案。



ToT特别适合解决以下高复杂度问题：

- 多方案决策（职业选择、投资策略、产品路线）
- 长程规划与策略制定
- 需要权衡多重约束的复杂推理
- 创造性问题求解（故事创作、方案设计）

其核心优势在于**显著提升了模型在不确定性和高风险场景下的决策质量**，同时输出的可解释性极强——整个思考过程像一棵清晰的决策树，用户可以清楚看到模型"想了哪些路、为什么放弃某些路、最终选择了哪条路"。

思维树的运行过程由以下四个关键动作组成，每个动作都有明确的操作方式和目的：

1. **生成初始分支**：模型同时从多个不同核心视角生成多条独立的初始思考路径，确保探索空间足够广阔，避免单一思路的局限性。
2. **自我评估**：模型对每条初始分支进行独立、客观的评估。通常采用多维度打分，综合判断该路径的可行性、潜在收益、风险水平以及与最终决策目标的匹配度，并给出简要理由。
3. **动态剪枝**：根据自我评估结果，模型主动淘汰得分较低或明显不可行的分支，只保留最具潜力的少数路径继续探索，从而避免在低价值路径上浪费资源，提高整体效率。
4. **扩展与汇聚**：对保留的高潜力分支进行进一步深入扩展，加入更多约束条件和细节分析；最后将所有保留路径进行对比，收敛到整体最优的一条路径，并输出明确的最终结论和详细理由。

通过这四个动作的有序执行，思维树实现了"发散探索—精准评估—高效剪枝—深度收敛"的完整闭环，是处理高复杂度决策问题的强大方法。

3.6.2 相关理论基础

思维树（ToT）的理论基础主要来自以下三个层面：

1.**搜索算法理论（Search Algorithms）**

ToT直接借鉴了人工智能中的经典搜索思想（如蒙特卡洛树搜索 MCTS、束搜索 Beam Search）。它把问题求解过程建模为**状态空间搜索**：每个思考步骤是一个"节点"，不同推理方向形成"分支"，模型通过自我评估来决定哪些分支值得继续扩展。这与AlphaGo在围棋中使用的"树搜索+策略网络"本质上一致，只是把"棋步"换成了"语言推理步骤"。

2.**人类认知科学与问题解决理论**

ToT模仿了人类专家在面对复杂问题时的真实思考模式——我们很少只走一条路，而是会同时脑中闪现多个方案、快速评估优劣、放弃明显不靠谱的想法、再对有潜力的方案深入分析。ToT正是把这种"并行思考 + 自我反思"的认知过程显式化，让模型也能进行类似的高级认知活动。

3.**大语言模型的能力演进与规划理论**

早期研究发现，单纯的CoT虽然能提升推理，但模型在长程规划和错误恢复能力上仍然较弱（容易在一条错误路径上越走越远）。ToT的提出正是为了弥补这一短板。它将LLM的**生成能力**（branching）和**评估能力**（self-evaluation）结合起来，实现了"生成-评估-搜索"的闭环，使模型从"被动跟随提示"进化到"主动规划求解"。

3.6.3 具体实现

核心思想：模型像"树"一样同时探索**多个思考路径**，然后评估每条路径，最终选择最佳答案。
比 CoT 更强大，适合需要多方案比较的任务。

```
from langchain_core.prompts import PromptTemplate

question = "写一个关于未来AI与人类共存的短篇小说大纲，要求包含冲突和情感深度。"

tot_prompt = PromptTemplate.from_template(
    """角色（Role）：你是一个擅长故事创作的资深小说家。
```

```
任务（Task）：使用 Tree of Thoughts（思维树）方法，为用户创作小说大纲。

内容（Context）：{question}

约束（Constraint）：
请严格按照以下 Tree of Thoughts 流程进行：
1. 先生成多个候选思路（至少3个）。
2. 对每个思路进行评估（优点、缺点、可行性）。
3. 删除明显较差的思路。
4. 对保留下来的思路继续扩展新的子思路。
5. 每次扩展后再次评估。
6. 当某条路径已经形成完整方案时停止扩展。
7. 输出完整的思维树过程。
8. 最终选择最优方案并说明选择理由。

输出格式（Output Format）：
请按以下结构输出：

【初始候选思路】
思路1：...
思路2：...
思路3：...

【第一轮评估与筛选】
（评估每个思路，并说明删除或保留原因）

【扩展与深入】
（对保留的思路进行扩展，生成子思路）

【最终方案】
推荐方案：思路X
理由：...
完整大纲：..."""
)

prompt = tot_prompt.format(question=question)
print("=== ToT 提示词 ===\n", prompt)

llm_output = llm.invoke(prompt, temperature=0.7)
print("\n模型输出（思维树）：\n", llm_output)
```

3.6.4 思维树 vs 思维链（CoT）

特性	思维树（ToT）	思维链（CoT）
推理结构	树形（多路径并行）	线性（单路径顺序）
灵活性	动态调整路径，支持回溯	固定顺序，难以修正早期错误
计算成本	较高（需维护多路径状态）	较低（单路径逐步生成）

特性	思维树（ToT）	思维链（CoT）
适用场景	复杂问题（如数学证明、游戏策略）	简单推理（如算术计算）

3.7 CoT、生成知识提示、Self-Consistency、ToT 对比总结

3.7.1 核心思想对比

技术	核心思想	推理结构	是否多路径	是否有评估/筛选	主要改进方向
CoT	强制模型一步一步思考	线性链	否	否	让模型显式展示推理过程
生成知识提示	先让模型生成相关背景知识，再基于知识回答	两阶段（生成+回答）	否	否	增强模型的知识调用能力
Self-Consistency	生成多条推理路径，取多数一致的答案	多条线性链	是	简单多数投票	降低单条路径出错风险
ToT (Tree of Thoughts)	像树一样探索多条路径，评估、剪枝、扩展	树状结构	是	有评估和剪枝	更系统化的搜索与规划

3.7.2 适用场景对比

场景	推荐技术	理由说明	不推荐原因
简单多步计算题	**CoT**	够用，成本最低	-
需要背景知识的问题	**生成知识提示**	先让模型自己“回忆”相关知识，再回答，效果更好	CoT 容易遗漏关键知识
数学、逻辑推理题	**Self-Consistency**	多条路径验证能有效减少计算错误	ToT 成本过高
需要多个方案对比的问题	**ToT**	支持生成、评估、筛选多条思路，适合决策类任务	Self-Consistency 探索不足
创意写作 / 故事构思	**ToT**	能系统地探索不同剧情方向、人物设定和结局	CoT 和 Self-Consistency 发散性不足
高准确率要求 + 可接受成本	**Self-Consistency**	在准确率和成本之间取得较好平衡	-
极复杂规划 / 搜索问题	**ToT**	具备评估、剪枝、回溯能力，探索空间更大	计算成本高

3.7.3 技术演进关系与选择建议

```
CoT（基础）
    ├── 生成知识提示（针对知识不足场景）
    ├── Self-Consistency（CoT 的多路径增强版）
    └── ToT（更强大的树状搜索版本）
```

选择建议（按复杂度排序）：

任务复杂度	推荐优先级	说明
低	CoT > 生成知识提示	够用就行
中	Self-Consistency	准确率与成本平衡最好
高	ToT	需要深度探索和规划时使用
极高	ToT + Self-Consistency 结合	高级用法（可先用 ToT 探索，再用 Self-Consistency 验证）

4.Output 输出

4.1 结构化输出

4.1.1 Json 格式介绍

1. 什么是 JSON?

JSON（JavaScript Object Notation，JavaScript 对象标记法）是一种**轻量级的数据交换格式。**它最初是为 JavaScript 语言设计的，但如今已经成为一种**与编程语言无关**的通用数据格式，被广泛用于 Web 开发、配置文件、API 数据传输、数据库存储等场景。

简单来说：

JSON 就是一个用纯文本写的"结构化数据文件"，它能让计算机程序之间轻松地传递和读取复杂的数据。

2. JSON 的核心特点

- **易读易写**：人类可以直接看懂和编辑（不像二进制文件）。
- **轻量级**：相比 XML 等其他格式，JSON 的语法更简洁，文件体积更小。
- **跨语言支持**：几乎所有主流编程语言（Python、Java、JavaScript、C#、Go、PHP 等）都原生支持 JSON 的解析和生成。
- **结构清晰**：用**键值对（Key-Value）**组织数据，层次分明。

3. JSON 的基本语法规则

一个标准的 JSON 文件必须满足以下规则：

- **数据结构**：
    - **对象（Object）**：用大括号 `{}` 包裹，里面是"键: 值"对。
    - **数组（Array）**：用方括号 `[]` 包裹，可以包含多个值或对象。

- **键（Key）**：必须是**双引号**包裹的字符串（不能省略引号）。
- **值（Value）** 支持以下 6 种数据类型：
    - 字符串（string）：用双引号 `""` 包裹
    - 数字（number）：整数或浮点数
    - 布尔值（boolean）：`true` 或 `false`
    - 空值（null）
    - 对象（object）
    - 数组（array）
- **其他注意事项**：
    - 键值对之间用**逗号** `,` 分隔（最后一个键值对后不能有逗号）。
    - 字符串中不能直接换行（可用 `\n` 表示）。
    - JSON 文件通常以 `.json` 为扩展名。

4. 简单例子

示例1：一个商品列表的 JSON 数组

```
item = [
  {
    "id": 1,
    "product": "iPhone 16",
    "price": 7999,
    "inStock": true
  },
  {
    "id": 2,
    "product": "AirPods",
    "price": 1299,
    "inStock": false
  }
]
```

示例2：Json的保存和加载

```
import json
with open('item.json', 'w', encoding='utf-8') as f:
    json.dump(student, f, ensure_ascii=False, indent=4)

print("✅ item.json 文件已保存！")
```

参数介绍：

- ensure_ascii=False：让中文正常显示（不转成 \u4e2d\u6587）
- indent=4：让文件排版漂亮（缩进 4 个空格），方便人阅读

```python
with open('item.json', 'r', encoding='utf-8') as f:
    data = json.load(f)   # 直接得到 dict

print(type(data))        # <class 'dict'>
```

4.1.2 输出 json 格式

现在我们想要让大模型输出的内容变为结构化数据，可以修改提示词

```python
# 尝试嵌入JSON结构
broken_template = """
你是一个天气查询助手。请根据以下格式返回天气信息：
{
    "city": "{city}",
    "date": "{date}",
    "weather":
}
"""

prompt = PromptTemplate(input_variables=["city", "date"], template=broken_template)
print(prompt.format(city="北京", date="2025-6-18"))
```

却发现结果报错了，查看报错代码是，`KeyError: '\n    "city"'`

简单来说，由于我们想输出json格式的内容，但是最外层的{}会让模板认为里面都是变量。

- 解决方案1：修改json结构的{}

```
...
{{
    "city": "{city}",
    "date": "{date}",
    "weather":
}}
...
```

这种方法适用于少量特殊符号的结构

- 解决方案2：变量注入

```python
# 将JSON作为变量整体注入
fixed_template = """
你是一个天气查询助手。请根据用户提供的城市和日期
城市：{city}
日期：{date}
并根据以下格式返回天气信息：
格式：
{params}
"""

```

```
prompt = PromptTemplate(
    input_variables=["city", "date", "params"],
    template=fixed_template
)

# 生成时传入已转义的JSON字符串
final_prompt = prompt.format(
    city="北京",
    date="2025-06-18",
    params='''{
        "city": "北京",
        "date": "2025-06-18",
        "weather":
    }'''
)
print(final_prompt)
```

4.2 输出封装

在 LangChain 框架的 Model I/O 模块中，“O” 指 **Output Parsers（输出解析器）**。

输出解析器是 LangChain 专门设计用于处理大语言模型（LLM）返回结果的核心组件。其主要功能是：

- 将 LLM 生成的原始、非结构化文本（string）转换为结构化数据（如 Python 对象、JSON、Pydantic 模型等）；
- 在提示词（Prompt）中自动注入格式要求，指导模型按指定格式输出；
- 提供 `parse()` 方法，实现自动解析与校验，确保输出结果可直接被下游代码使用。

Output Parsers 是构建可靠 Agent、RAG 系统、自动化工作流时不可或缺的桥梁，它将“人类可读的文本”转化为“机器可编程的结构化数据”。

4.2.1 原因

LLM 的原始输出存在以下固有问题，必须通过 Output Parsers 解决：

1. **非结构化**：模型返回的是连续的自然语言文本，无法直接提取字段（如名称、日期、列表等）。
2. **格式不一致**：即使要求模型输出 JSON，实际结果仍可能出现多余说明、格式错误或遗漏，导致程序解析失败。
3. **可编程性差**：后续代码难以可靠地处理原始字符串，容易引发 bug 或需要大量自定义正则/字符串处理逻辑。
4. **稳定性要求高**：在生产级应用（如智能体、数据提取、表单填充）中，需要严格的类型校验和错误处理，而 Output Parsers 可自动完成这些工作，提升整个链路的可靠性和可维护性。

因此，Output Parsers 的核心价值在于：**让 LLM 输出从“不可控的文本”变成“可信的结构化数据”**，是 LangChain 实现复杂应用的关键基础设施。

4.2.2 常见解析器

LangChain 提供了多种 Output Parsers，以满足不同场景下的结构化需求。以下是常用解析器的对比总结表：

解析器名称	描述	适用场景	备注
StrOutputParser	最基础的解析器，直接返回清理后的字符串文本	纯文本生成任务，无需结构化	最基础类型
JsonOutputParser	将输出解析为标准 JSON 对象	需要提取键值对的结构化数据任务	-
CommaSeparatedListOutputParser	将逗号分隔的文本解析为 Python 列表	关键词提取、标签列表、简单枚举项	简单实用

通过上述解析器，开发者可大幅降低对 LLM 输出格式的手动处理工作量，确保讲义后续章节中 Agent、RAG 等高级应用能够稳定运行。

4.2.3 示例：StrOutputParser

```python
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# 1. 最简单的 PromptTemplate
prompt_template = PromptTemplate.from_template(
    input_variables=["topic"],
    template="""
任务（Task）：请用中文给主题写一段简短介绍，控制在50字以内。
内容（Context）：主题：{topic}
"""
)

# 2. 格式化提示词
prompt = prompt_template.format(topic="大模型")
print("发送给模型的提示词：\n", prompt)

# 3. 真实调用你的模型
llm_output = llm.invoke(prompt)           # ← 直接用你的 llm
print("\n模型原始输出：", llm_output)

# 4. 解析
output_parser = StrOutputParser()
result = output_parser.parse(llm_output)

print("\n解析后结果类型：", type(result))
print("解析后结果：", result)
```

不同的 LLM 类、不同的调用方式，返回的对象类型可能不同：

- 有的返回纯字符串（`str`）

- 有的返回 `AIMessage` 对象（包含 `content`、`tool_calls` 等字段）
- 有的返回 `BaseMessage` 列表

StrOutputParser 的核心职责：从上述任何一种返回类型中，**可靠地提取出最终的字符串内容**，确保下游代码总是拿到 `str` 类型。在后面讲解到 LCEL 之后，会再次进行讲解。

4.2.4 示例：CommaSeparatedListOutputParser

使用场景： CommaSeparatedListOutputParser 适合让模型返回一个**列表**（如关键词、标签、步骤、推荐项等），它会自动把逗号分隔的文本解析成 Python 的 list，方便后续循环处理。

```python
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# 1. 最简单的 PromptTemplate
prompt_template = PromptTemplate.from_template(
    input_variables=["topic"],
    template="""
任务（Task）：列出5个与'{topic}'相关的热门关键词，用英文逗号分隔。。
内容（Context）：主题：{topic}
"""
)

# 2. 格式化提示词
prompt = prompt_template.format(topic="大模型")
print("发送给模型的提示词：\n", prompt)

# 3. 真实调用你的模型
llm_output = llm.invoke(prompt)           # ← 直接用你的 llm
print("\n模型原始输出：", llm_output)

# 4. 解析
output_parser = CommaSeparatedListOutputParser()
result = output_parser.parse(llm_output)

print("\n解析后结果类型：", type(result))
print("解析后结果（Python 列表）：", result)
```

- `CommaSeparatedListOutputParser` 将 LLM 返回的逗号分隔字符串（如 `"A, B, C"`）自动解析成 Python 列表（如 `['A', 'B', 'C']`）。
- 省去手动 `split()` 和清理空格的麻烦，直接获得可遍历、可索引的列表类型。
- 适用于关键词提取、标签列表等简单结构化场景，让代码更简洁可靠。

4.2.5 示例：JsonOutputParser

使用场景： JsonOutputParser 适合让模型返回结构化数据（如学生信息、商品详情、配置参数、API 返回结果等），它会自动把文本解析成 Python 的 dict 或 list，可以直接 json.dump() 保存文件或进行数据操作。

```python
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate

# 1. 构造带约束、任务、内容的 PromptTemplate
prompt_template = PromptTemplate.from_template(
    input_variables=["topic"],
    template="""
任务（Task）：根据主题生成一个商品信息
内容（Context）：主题：{topic}
约束（Constraint）：必须严格返回JSON格式，不要加任何其他文字和解释。
"""
)

# 2. 格式化提示词
prompt = prompt_template.format(topic="智能手机")
print("发送给模型的提示词：\n", prompt)

# 3. 真实调用你的模型
llm_output = llm.invoke(prompt)           # ← 直接用你的 llm
print("\n模型原始输出：", llm_output)

# 4. 解析
output_parser = JsonOutputParser()
result = output_parser.parse(llm_output)

print("\n解析后结果类型：", type(result))
print("解析后结果（Python 字典）：")
print(result)
print("商品名称：", result["name"])
```

- 将 LLM 返回的 JSON 字符串（如 `{"name": "智能手机", "price": 3999}`）自动解析为 Python 字典，可直接通过键名取值（如 `result["name"]`）。
- 省去手动 `json.loads()` 和异常处理，同时通过提示词中的“约束”强制模型只输出纯 JSON，避免额外文字干扰解析。
- 适用于需要提取结构化数据（如商品信息、用户档案、表单内容）的场景，让数据直接可编程使用。